

ReelMind

Phase 1 — Complete Build Plan

iOS (Swift + SwiftUI) · 44 micro-tasks across 4 layers · Build in sequence top to bottom

Layer 1 — Infrastructure & Plumbing

Build this first. Nothing else works without it.

- | | | |
|----|----------|---|
| 1 | Xcode | Create Xcode project
New iOS app target, SwiftUI lifecycle, deployment target iOS 16+. |
| 2 | Xcode | Add Share Extension target
File → New Target → Share Extension. This is the core save mechanism for capturing reel URLs. |
| 3 | Xcode | Configure App Groups entitlement
Enable App Groups on both the main app and the Share Extension so they can share a container to pass URLs between them. |
| 4 | Supabase | Set up Supabase project
Project is already created. Add URL + anon key (JWT) into a Config.swift constants file. Do NOT commit to Git. |
| 5 | Supabase | Design database schema
Tables: users, reels (url, transcript, category, confidence, creator_handle, thumbnail_url, status, deleted_at), categories (name, is_default, user_id). |
| 6 | Supabase | Enable pgvector extension
Run: CREATE EXTENSION vector; in Supabase SQL editor. Add vector(1536) column to reels table for embeddings. |
| 7 | Supabase | Configure Supabase Auth + RLS
Enable Email + Google Sign-In providers. Add row-level security policies so users only see and modify their own data. |
| 8 | Backend | Scaffold FastAPI backend
Python project: fastapi, uvicorn, supabase-py, openai, celery, redis. Create /api/v1/ route skeleton with health check. |
| 9 | Backend | Set up Redis + Celery workers
Redis as message broker. Celery handles async transcription, embedding, and classification jobs so the API returns instantly. |
| 10 | Infra | Deploy backend to Railway or Render
Connect GitHub repo, set env vars (SUPABASE_URL, SUPABASE_KEY, OPENAI_KEY, REDIS_URL, FCM_KEY). Get a stable live API URL. |

11	Push	Configure Firebase Cloud Messaging Create Firebase project, add iOS app, download GoogleService-Info.plist, add to Xcode. Enable push notification capability.
----	------	--

Layer 2 — Core AI Pipeline

The actual product. This is what makes ReelMind magical.

12	Swift	Build Share Extension capture flow Minimal SwiftUI extension UI: "Saving..." spinner. On activate, extract URL from NSExtensionItem and write to App Group shared container.
13	Swift	POST URL to backend on share Extension calls POST /api/v1/reels with the reel URL + user auth token. Extension dismisses immediately — no waiting for processing.
14	Backend	Queue ingestion job in Celery API receives URL → creates reel record (status: queued) → dispatches Celery task → returns HTTP 202 immediately.
15	Backend	Download reel audio in worker Evaluate a third-party reel downloader library (check ToS + reliability). Extract audio as mp3. Handle: private, deleted, region-blocked URLs.
16	AI	Transcribe audio with Whisper Call openai.audio.transcriptions.create(model='whisper-1'). Store full transcript text on the reel record in Supabase.
17	AI	Fallback: caption + hashtag extraction If Whisper returns empty transcript (music-only reel), extract caption text and hashtags from reel metadata to use as classification input.
18	AI	LLM classification prompt Send transcript + caption to Claude. Prompt returns JSON: {category, confidence, summary}. Support all 6 default categories + any user-created ones.
19	Backend	Confidence threshold routing Score >= 0.75: auto-assign to category. Score < 0.75: route to Uncategorised inbox. Update reel record status accordingly.
20	AI	Chunk + embed transcript Split transcript into ~200 token chunks. Call text-embedding-3-small on each chunk. Store vectors in pgvector column in Supabase.
21	Backend	Duplicate URL detection Before processing, check if URL already exists for this user. If yes, skip re-processing and notify: "You already saved this on [date]."

22	Push	Send FCM push on completion Fire push: "Saved to Skincare - tap to reassign". Include category and reel_id in notification payload data for the app to act on.
23	Swift	Handle notification inline reply Register UNNotificationCategory with a text input action. On reply, call PATCH /api/v1/reels/{id}/category with the typed category name.
24	Backend	Auto-create category on reply Backend: if category doesn't exist for this user, create it. Move the reel. Send confirmation push: 'Added to Makeup'.

Layer 3 — App UI & Library View

Build views on top of already-working data.

25	SwiftUI	Authentication screens SwiftUI login + signup screens. Use Supabase Auth Swift SDK. Store session token in Keychain. Share token with Share Extension via App Group.
26	SwiftUI	Home screen — category pills Horizontal ScrollView of pills: 6 defaults (Skincare, Haircare, Bodycare, Fitness, Nutrition, Fashion) + user-created categories + Uncategorized badge showing count.
27	SwiftUI	Reel card grid per category On category tap, fetch reels from Supabase filtered by category_id. Display: thumbnail, creator handle, 2-line auto-summary, date saved.
28	SwiftUI	Thumbnail caching Store thumbnails in Supabase Storage. Load via AsyncImage with URLCache disk caching to avoid re-fetching on every scroll.
29	SwiftUI	Long-press context menu On card long-press: sheet with Reassign Category option, Delete with confirmation alert, Open in Instagram deep link (instagram.com/reel/{id}).
30	SwiftUI	Uncategorised inbox view Badge visible on home screen when count > 0. Tapping opens a list of unassigned reels, each with a prominent "Assign Category" CTA button.
31	SwiftUI	Delete reel flow Soft delete: set deleted_at timestamp in Supabase. Also delete vectors from pgvector for that reel. Show confirmation alert before deleting.
32	SwiftUI	Reassign category bottom sheet Bottom sheet shows list of existing categories + "New category" text input. On confirm, updates reel record and refreshes the grid.

33	SwiftUI	Graceful degradation card state If Watch Reel URL returns an error, show "Content may be unavailable" on the card. Always retain the transcript for query purposes.
34	SwiftUI	Empty + first-use onboarding states First launch: full empty state with illustrated tip showing how to share a reel to the app. Per-category empty state: "Save a reel here to get started."

Layer 4 — Query Layer (The Magic)

Build last — all transcript data must be in the DB first.

35	Backend	Build RAG retrieval endpoint POST /api/v1/query. Embed user question via text-embedding-3-small. Run pgvector similarity search filtered by category_id. Return top-5 chunks + reel metadata.
36	Backend	Web search fallback logic If fewer than 2 chunks score above 0.7 similarity threshold, call Tavily API. Append results to LLM context. Tag all web results clearly as "web source".
37	AI	LLM synthesis call Assemble context (reel chunks + optional web results). Call Claude with strict RAG prompt: answer only from provided context, cite each claim to a source.
38	Backend	Stream response to iOS client Use Server-Sent Events (SSE) to stream LLM response tokens to iOS as they generate. App renders text progressively as it arrives.
39	SwiftUI	Per-category chat screen "Ask" button on each category screen. Opens a dedicated chat view scoped to that category. Text input bar at bottom. Messages scroll upward.
40	SwiftUI	Rich card rendering below response After the AI chat message, render 2-4 result cards. Each card: reel thumbnail, creator handle, 2-line transcript extract, Watch Reel deep-link button.
41	SwiftUI	Watch Reel deep link "Watch Reel" button opens URL: https://instagram.com/reel/{shortcode} . Opens Instagram app if installed; falls back to Safari.
42	SwiftUI	Source attribution labels Each result card is clearly labelled "From your library" or "From the web". Web-sourced cards show the source domain instead of a creator handle.
43	SwiftUI	Conversation session state Maintain full message history in memory per chat session. Pass complete history to each LLM call so follow-up questions work naturally.

44

SwiftUI

Thumbs up / down on responses

Feedback buttons (+ / -) on each AI response. POST /api/v1/feedback with response_id and rating. Used for quality metrics from PRD Section 8.